

MonetDB Complete

Datatypes:

```
CREATE TABLE emp_reg (  
  id      INT,           -- integer  
  age     SMALLINT,      -- small integer  
  salary  DECIMAL(5,2),   -- fixed precision decimal  
  initial CHAR(1),       -- single character  
  name    VARCHAR(50),    -- variable length string  
  dt      DATE,          -- calendar date  
  tm      TIME,          -- clock time  
  profile JSON,          -- JSON document  
  ip      INET,          -- IP address  
  sid     SERIAL,        -- auto increment ID  
  link    URL,           -- URL link  
  uid     UUID           -- universally unique identifier  
);
```

```
INSERT INTO emp_reg (  
  id, age, salary, initial, name, dt, tm, profile, ip, link, uid  
)  
VALUES (  
  1,           -- INT  
  25,         -- SMALLINT  
  55000.75,    -- DECIMAL(5,2)  
  'A',        -- CHAR(1)  
  'Arjun Patel', -- VARCHAR  
  DATE '2000-05-15', -- DATE  
  TIME '09:30:00',  -- TIME  
  '{"skills":["SQL","Python"],"exp":3}', -- JSON  
  '192.168.1.10',   -- INET  
  'https://www.example.com/profile/arjun', -- URL  
  SYS.UUID()        -- UUID (auto-generated unique ID)  
);
```

Case Sensitive:

1. SQL Keywords (queries): Keywords are *not* case-sensitive.

Example: `SELECT * FROM emp_reg;`

You can also write `select * from emp_reg;` or `SeLeCt * FrOm emp_reg;` — all work the same way.

2. Identifiers (table names, column names): By default, table and column names are stored in lowercase. If you create "Emp_Test" and "Name" (with quotes), then you must always write them exactly with the same case and quotes: `SELECT "Name" FROM "Emp_Test";`

3. String and JSON values: String values are case-sensitive.

Example: If you inserted 'Arjun Patel' into the name column, a query where name = 'Arjun Patel' works, but where name = 'arjun patel' does not match anything.

JSON is also case-sensitive. '{"skills":["SQL"]}' is different from '{"Skills":["SQL"]}'. So you must match the key name exactly.

Schema Commands

Keyword	Description	Example
CREATE SCHEMA	Create a new logical container for tables, views, sequences, etc. Supports IF NOT EXISTS and optional AUTHORIZATION.	CREATE SCHEMA IF NOT EXISTS university;
COMMENT ON SCHEMA	Add or remove a human-readable comment (use NULL or " to remove).	COMMENT ON SCHEMA university IS 'Teaching DB for labs';
SET SCHEMA	Switch the current schema; new objects without qualifiers are created here.	SET SCHEMA university;
CURRENT_SCHEMA	Retrieve the name of the current schema (handy to verify SET SCHEMA).	SELECT CURRENT_SCHEMA;
Fully-qualified names	Create/read objects in another schema by prefixing with schema.object.	CREATE TABLE university.courses(id INT);
AUTHORIZATION	Create a schema owned by a user/role (ownership fixed after creation).	CREATE ROLE fac; CREATE SCHEMA "HR" AUTHORIZATION fac;
INFORMATION_SCHEMA	List non-system schemas via SQL standard view.	SELECT schema_name FROM information_schema.schemata WHERE NOT is_system;
ALTER SCHEMA RENAME	Rename a schema (only if no external dependencies).	ALTER SCHEMA university RENAME TO university_v2;
DROP SCHEMA RESTRICT	Drop only if the schema is empty (switch away from it first).	SET SCHEMA sys; DROP SCHEMA university RESTRICT;
DROP SCHEMA CASCADE	Drop schema and all contained objects (tables, views, sequences, etc.).	SET SCHEMA sys; DROP SCHEMA IF EXISTS university CASCADE;

Table commands:

Keyword	Description	Example
CREATE TABLE	Create a new regular table in the current schema; table names are unique within schema.	CREATE TABLE students (sid INT PRIMARY KEY, name VARCHAR(100) NOT NULL); INSERT INTO students (sid, name) VALUES (101,'gopi'); INSERT INTO students (sid, name) VALUES (102,'krishnan');
CREATE TABLE AS (CTAS)	Create a table based on a query result. WITH DATA copies rows; WITH NO DATA copies only structure.	CREATE TABLE student_names AS SELECT name FROM students WITH DATA;
CREATE TEMP TABLE	Create session-scoped tables stored under schema tmp. LOCAL visible only in session; GLOBAL defined in catalog but data isolated.	CREATE TEMP TABLE tmp_scores(sid INT, score INT) ON COMMIT PRESERVE ROWS;
CREATE MERGE TABLE	Define a virtual table combining partitions with identical structure. Query acts like a UNION.	CREATE MERGE TABLE marks_all (sid INT, course VARCHAR(30), mark INT); ALTER TABLE marks_all ADD TABLE marks_2024;
CREATE REMOTE TABLE	Create an alias to a table/view on another MonetDB server. Names and types must match exactly.	CREATE REMOTE TABLE remote_students (sid INT, name VARCHAR(100)) ON 'mapimonetdb://host:50000/otherdb' WITH USER 'usr' ENCRYPTED PASSWORD 'pwd';
CREATE UNLOGGED TABLE	Create a table without write-ahead logging; faster but non-durable. Can optionally be INSERT ONLY.	CREATE UNLOGGED TABLE foo(x INT);
DROP TABLE	Remove a table from schema. RESTRICT requires no dependents, CASCADE also drops dependent objects.	DROP TABLE IF EXISTS university.student_names CASCADE;
INFORMATION_SCHEMA & sys.tables	Inspect user tables, columns, and stats via system catalogs and views.	SELECT * FROM information_schema.tables WHERE table_schema='university';

Keyword	Description	Example
ALTER TABLE ... ADD COLUMN	Add a new column branch of type VARCHAR(50) to the student table.	ALTER TABLE student ADD COLUMN branch VARCHAR(50);
ALTER TABLE ... RENAME COLUMN	Rename the column name to sname for clarity.	ALTER TABLE student RENAME COLUMN name TO sname;
Add new integer column	Adds a new column sno of type INTEGER to the table	ALTER TABLE student ADD COLUMN sno INTEGER; ALTER TABLE student ADD COLUMN sem INTEGER DEFAULT 5;
Step 1 — Add new column with desired type	Creates a temporary column sno_new with the target data type DECIMAL(5,2)	ALTER TABLE student ADD COLUMN sno_new DECIMAL(5,2);
Step 2 — Copy existing values	Copies and converts values from the old column sno into sno_new using MonetDB's CAST()	UPDATE student SET sno_new = CAST(sno AS DECIMAL(5,2));
Step 3 — Drop the old column	Removes the previous integer column once data has been safely copied	ALTER TABLE student DROP COLUMN sno;
Step 4 — Rename new column	Renames sno_new back to sno to keep the original name with the new type	ALTER TABLE student RENAME COLUMN sno_new TO sno;
DESCRIBE	Displays table definition: column names, data types, and nullability. Useful to check schema after modifications.	DESCRIBE student; \d student

Key constrains:

Feature	Example Command	Explanation
Basic table	CREATE TABLE student (sid INT, regno VARCHAR(20), name VARCHAR(100), dob DATE);	Simplest form, defines 4 columns with types, no constraints.
PRIMARY KEY	CREATE TABLE student (regno VARCHAR(20) PRIMARY KEY, name VARCHAR(100), dob DATE);	regno is unique and non-null; acts as the identifier.
NOT NULL	CREATE TABLE student (regno VARCHAR(20) PRIMARY KEY, name VARCHAR(100) NOT NULL, dob DATE);	name cannot be left empty on inserts.
UNIQUE	CREATE TABLE student (sid INT PRIMARY KEY, regno VARCHAR(20) UNIQUE, name VARCHAR(100), dob DATE);	sid is PK, but regno must still be unique.
CHECK constraint	CREATE TABLE student (sid INT PRIMARY KEY, name VARCHAR(100), cgpa DECIMAL(4,2) CHECK (cgpa BETWEEN 0 AND 10));	Validates CGPA values; anything outside 0–10 rejected.
DEFAULT value	CREATE TABLE student (sid INT PRIMARY KEY, regno VARCHAR(20), name VARCHAR(100) NOT NULL, cgpa DECIMAL(4,2) DEFAULT 0.0);	Inserts without cgpa auto-fill 0.0.
FOREIGN KEY	CREATE TABLE department (did INT PRIMARY KEY, dname VARCHAR(50)); CREATE TABLE student (sid INT PRIMARY KEY, regno VARCHAR(20) UNIQUE, did INT, FOREIGN KEY (did) REFERENCES department(did));	Links student.did to department.did, enforcing referential integrity.
Using SEQUENCE	CREATE SEQUENCE sid_seq START WITH 101; CREATE TABLE student (sid INT PRIMARY KEY DEFAULT NEXT VALUE FOR sid_seq, regno VARCHAR(20) UNIQUE, name VARCHAR(100) NOT NULL, dob DATE);	Auto-increments sid using an explicit sequence.
Using SERIAL	CREATE TABLE student (sid SERIAL PRIMARY KEY, regno VARCHAR(20) UNIQUE, name VARCHAR(100) NOT NULL, dob DATE);	Shorthand: SERIAL auto-creates and binds a sequence.

Sequences

Keyword	Description	Example	Output
CREATE SEQUENCE	Defines an auto-increment generator for student IDs, starting from 101 and incrementing by 1.	CREATE SEQUENCE sid_seq START WITH 101 INCREMENT BY 1;	operation successful
CREATE TABLE with DEFAULT	Creates student table, uses DEFAULT NEXT VALUE FOR sid_seq so sid auto-fills from sequence.	CREATE TABLE student (sid INT PRIMARY KEY DEFAULT NEXT VALUE FOR university.sid_seq, regno VARCHAR(20) NOT NULL, name VARCHAR(100) NOT NULL, dob DATE NOT NULL, marks INT, cgpa DECIMAL(4,2), address JSON);	operation successful
INSERT row (auto sid)	Insert values without sid; MonetDB fills it from sequence automatically.	INSERT INTO student (regno, name, dob, marks, cgpa, address) VALUES ('25CSE001','Arjun Patel',DATE '2005-04-12',86, 8.75,CAST('{"street":"123 Elm St","city":"New York","state":"NY"}' AS JSON));	1 row inserted
SELECT data	Retrieve inserted record, showing sequence-assigned sid (101).	SELECT * FROM student;	sid=101, regno=25CSE001, name=Arjun Patel, dob=2005-04-12, cgpa=8.75, address={"street":"123 Elm St","city":"New York","state":"NY"}

Consider this table:

regno	sname	dob	cgpa	address	branch
25001	Rohan Mehta	3/15/2005	8.1	{"street":"MG Road","city":"Mumbai","state":"MH"}	CSE
25002	Ananya Gupta	7/22/2006	8.9	{"street":"Park Street","city":"Kolkata","state":"WB"}	ECE
25003	Vikram Reddy	11/5/2005	7.2	{"street":"Jubilee Hills","city":"Hyderabad","state":"TS"}	EEE
25004	Priya Nair	1/12/2006	9.3	{"street":"MG Road","city":"Bengaluru","state":"KA"}	IT
25005	Karthik Sharma	9/25/2005	6.85	{"street":"Civil Lines","city":"Delhi","state":"DL"}	MECH

```
CREATE SEQUENCE sid_seq START WITH 25001 INCREMENT BY 1;
```

```
CREATE TABLE student (regno INT PRIMARY KEY DEFAULT NEXT VALUE FOR university.sid_seq,  
sname VARCHAR(100) NOT NULL, dob DATE NOT NULL, cgpa DECIMAL(4,2), address JSON, branch  
VARCHAR(50));
```

```
DROP TABLE IF EXISTS university.student CASCADE;
```

```
DROP SEQUENCE sid_seq;
```

```
INSERT INTO student (sname, dob, cgpa, address, branch) VALUES ('Rohan Mehta', DATE '2005-03-15',  
8.10, CAST('{"street":"MG Road","city":"Mumbai","state":"MH"}' AS JSON), 'CSE');
```

```
INSERT INTO student (sname, dob, cgpa, address, branch) VALUES ('Ananya Gupta', DATE '2006-07-22',  
8.90, CAST('{"street":"Park Street","city":"Kolkata","state":"WB"}' AS JSON), 'ECE');
```

```
INSERT INTO student (sname, dob, cgpa, address, branch) VALUES ('Vikram Reddy', DATE '2005-11-05',  
7.20, CAST('{"street":"Jubilee Hills","city":"Hyderabad","state":"TS"}' AS JSON), 'EEE'), ('Priya Nair',  
DATE '2006-01-12', 9.30, CAST('{"street":"MG Road","city":"Bengaluru","state":"KA"}' AS JSON), 'IT'),  
('Karthik Sharma', DATE '2005-09-25', 6.85, CAST('{"street":"Civil Lines","city":"Delhi","state":"DL"}' AS  
JSON), 'MECH');
```

```
SELECT * FROM student;
```

Projections

Keyword	Description	Example
Select all columns	Project all attributes from the table.	SELECT * FROM student;
Select single column	Retrieve only student names.	SELECT sname FROM student;
Select multiple columns	Show student name and branch only.	SELECT sname, branch FROM student;
Rename column (alias)	Use alias for readability.	SELECT sname AS student_name, cgpa AS grade FROM student;
Select constants	Add a literal value in projection.	SELECT sname, 'India' AS country FROM student;
String functions	Apply string function in projection.	SELECT UPPER(sname) AS name_caps FROM student;
Arithmetic in projection	Calculate derived column.	SELECT sname, cgpa*10 AS cgpa_percent FROM student;
Substring projection	Extract year from date of birth.	SELECT sname, EXTRACT(YEAR FROM dob) AS birth_year FROM student;
JSON as string	Show city from JSON address (basic cast).	SELECT sname, CAST(address AS STRING) AS full_address FROM student; SELECT sname, address AS full_address FROM student;

Selections:

Keyword	Description	Example
WHERE with equality	Select students from branch CSE.	SELECT sname, branch FROM student WHERE branch = 'CSE';
WHERE with comparison	Find students with CGPA above 8.5.	SELECT sname, cgpa FROM student WHERE cgpa > 8.5; SELECT sname, cgpa FROM student WHERE cgpa >= 8.5; SELECT sname, cgpa FROM student WHERE cgpa < 8.5; SELECT sname, cgpa FROM student WHERE cgpa <= 8.5;

Keyword	Description	Example
WHERE with BETWEEN	Get students with CGPA between 7.0 and 9.0.	SELECT sname, cgpa FROM student WHERE cgpa >= 8 AND branch = 'CSE'; SELECT sname, cgpa FROM student WHERE cgpa >= 8 OR branch = 'CSE'; SELECT sname, cgpa FROM student WHERE cgpa BETWEEN 7.0 AND 9.0;
WHERE with IN	Select students from specific branches.	SELECT sname, branch FROM student WHERE branch IN ('CSE','ECE');
WHERE with LIKE	Find students whose names start with 'A'.	SELECT sname FROM student WHERE sname LIKE 'A%';
WHERE with IS NULL	Get students without a branch assigned.	SELECT sname FROM student WHERE branch IS NULL;
WHERE with JSON (string search)	Students from Bengaluru city.	SELECT sname, address FROM student WHERE CAST(address AS STRING) LIKE '%Bengaluru%';
WHERE with AND	Students from CSE branch with CGPA $\geq$ 8.0.	SELECT sname, branch, cgpa FROM student WHERE branch='CSE' AND cgpa $\geq$ 8.0;
WHERE with OR	Students from Delhi OR Hyderabad.	SELECT sname, address FROM student WHERE CAST(address AS STRING) LIKE '%Delhi%' OR CAST(address AS STRING) LIKE '%Hyderabad%';
NOT (negation)	Exclude rows that match a condition; inverse of equality or LIKE.	SELECT sname, branch FROM student WHERE NOT branch = 'CSE'; SELECT sname, branch FROM student WHERE branch != 'CSE';
NOT with LIKE	Exclude names starting with 'A'.	SELECT sname FROM student WHERE sname NOT LIKE 'A%';
NOT with IN	Exclude multiple values at once.	SELECT sname, branch FROM student WHERE branch NOT IN ('CSE','ECE');
ORDER BY (Ascending)	Displays students sorted by CGPA in ascending order (lowest first)	SELECT regno, sname, cgpa FROM student ORDER BY cgpa ASC;
ORDER BY (Descending)	Displays students sorted by CGPA in descending order (highest first)	SELECT regno, sname, cgpa FROM student ORDER BY cgpa DESC;

Keyword	Description	Example
LIMIT	Restricts output to a specific number of rows (top 3 students by CGPA)	SELECT regno, sname, cgpa FROM student ORDER BY cgpa DESC LIMIT 3;
OFFSET (Skip)	Skips the first n rows and displays the rest (skip top 2 highest CGPAs)	SELECT regno, sname, cgpa FROM student ORDER BY cgpa DESC OFFSET 2;
LIMIT with OFFSET	Combines both to display a specific range (3rd and 4th students by CGPA)	SELECT regno, sname, cgpa FROM student ORDER BY cgpa DESC LIMIT 2 OFFSET 2;

Aggregate Functions

Keyword	Description	Example
COUNT(*)	Counts number of rows in a table.	SELECT COUNT(*) AS total_students FROM student;
COUNT(col)	Counts non-NULL values in a column.	SELECT COUNT(branch) AS branch_count FROM student;
SUM(col)	Returns the sum of all values in a column.	SELECT SUM(cgpa) AS total_cgpa FROM student;
AVG(col)	Computes average value of a column.	SELECT AVG(cgpa) AS avg_cgpa FROM student;
MIN(col)	Returns minimum value in a column.	SELECT MIN(cgpa) AS min_cgpa FROM student;
MAX(col)	Returns maximum value in a column.	SELECT MAX(cgpa) AS max_cgpa FROM student;
GROUP BY	Aggregates rows grouped by a column.	SELECT branch, AVG(cgpa) FROM student GROUP BY branch;
HAVING	Applies condition after grouping.	SELECT branch, AVG(cgpa) FROM student GROUP BY branch HAVING AVG(cgpa) > 8.5;
DISTINCT	Aggregates unique values only.	SELECT COUNT(DISTINCT branch) AS distinct_branches FROM student;

UPDATE queries

Keyword	Description	Example
Basic update	Change branch of a single student.	UPDATE student SET branch='AIML' WHERE sid=101;
Update multiple columns	Modify more than one field at once.	UPDATE student SET cgpa=9.1, branch='DS' WHERE sname='Ananya Gupta';
Update with arithmetic	Increase CGPA by 0.5 for all students in CSE.	UPDATE student SET cgpa=cgpa+0.5 WHERE branch='CSE';
Update using string function	Convert branch names to uppercase.	UPDATE student SET branch=UPPER(branch);
Update JSON (replace whole)	Change city in address JSON by replacing whole object.	UPDATE student SET address=CAST({'street':"MG Road","city":"Chennai","state":"TN"}' AS JSON) WHERE sid=104;
Update JSON (string replace trick)	Replace city text without rewriting full JSON.	UPDATE student SET address=CAST(REPLACE(CAST(address AS STRING), '"city":"Delhi"', '"city":"Noida"') AS JSON) WHERE sid=105;
Conditional update with subquery	Set branch to IT for students below average CGPA.	UPDATE student SET branch='IT' WHERE cgpa < (SELECT AVG(cgpa) FROM student);

DELETE and DROP

Keyword	Description	Example
DELETE single row	Remove one student by sid.	DELETE FROM student WHERE sid=101;
DELETE by condition	Delete all students in MECH branch.	DELETE FROM student WHERE branch='MECH';
DELETE using range	Delete students with regno < 25003.	DELETE FROM student WHERE regno < 25003;
DELETE using subquery	Remove students below average CGPA.	DELETE FROM student WHERE cgpa < (SELECT AVG(cgpa) FROM student);
DELETE all rows	Clear table but keep structure.	DELETE FROM student;
TRUNCATE TABLE	Faster delete all rows (metadata reset).	TRUNCATE TABLE student;

Keyword	Description	Example
DROP TABLE	Permanently remove the table definition and data.	DROP TABLE student;
DROP TABLE IF EXISTS	Prevents error if table doesn't exist.	DROP TABLE IF EXISTS student;
DROP SCHEMA RESTRICT	Delete schema if empty only.	DROP SCHEMA university RESTRICT;
DROP SCHEMA CASCADE	Force delete schema + all objects.	DROP SCHEMA university CASCADE;

Date Time Functions

Keyword	Description	Example
CURRENT_DATE	Returns today's date.	SELECT CURRENT_DATE;
CURRENT_TIME	Returns current system time.	SELECT CURRENT_TIME;
CURRENT_TIMESTAMP	Returns current date and time.	SELECT CURRENT_TIMESTAMP;
EXTRACT(field FROM date)	Extract part of a date (YEAR, MONTH, DAY).	SELECT EXTRACT(YEAR FROM dob) AS birth_year FROM student;
Students born after a specific date	Retrieves students whose DOB is later than a given date	SELECT * FROM student WHERE dob > DATE '2005-12-31';
Students born before a specific date	Retrieves students whose DOB is earlier than a given date	SELECT * FROM student WHERE dob < DATE '2006-01-01';
Students born in 2005	Extracts students whose DOB year is 2005	SELECT * FROM student WHERE EXTRACT(YEAR FROM dob) = 2005;
Students born in a specific month & year (Jan 2006)	Filters students born in a given month & year	SELECT * FROM student WHERE EXTRACT(YEAR FROM dob)=2006 AND EXTRACT(MONTH FROM dob)=1;
Students sorted by DOB (ascending)	Oldest to youngest	SELECT * FROM student ORDER BY dob ASC;
Students sorted by DOB (most recent first)	Youngest to oldest	SELECT * FROM student ORDER BY dob DESC;
Count students born in 2006	Returns total students with DOB year = 2006	SELECT COUNT(*) AS total_students FROM student WHERE EXTRACT(YEAR FROM dob) = 2006;

Keyword	Description	Example
Students born between two dates	Lists students whose DOB is within a date range	SELECT * FROM student WHERE dob BETWEEN DATE '2005-01-01' AND DATE '2005-12-31';
Display name and age (years)	Approximates age by year difference	SELECT sname, (EXTRACT(YEAR FROM CURRENT_DATE) - EXTRACT(YEAR FROM dob)) AS age_years FROM student;
Students older than 18 years	Filters by age > 18 (year-diff approx)	SELECT * FROM student WHERE (EXTRACT(YEAR FROM CURRENT_DATE) - EXTRACT(YEAR FROM dob)) > 18;
Calculate age (full years) per student	Shows regno, sname, dob, computed age	SELECT regno, sname, dob, (EXTRACT(YEAR FROM CURRENT_DATE) - EXTRACT(YEAR FROM dob)) AS age_years FROM student;
Students older than 19 years	Filters students with age > 19 (year-diff)	SELECT regno, sname, dob FROM student WHERE (EXTRACT(YEAR FROM CURRENT_DATE) - EXTRACT(YEAR FROM dob)) > 19;

CAST FUNCTIONS

Keyword	Description	Example
CAST to STRING	Convert a numeric column to string for display or pattern search.	SELECT sname, CAST(cgpa AS STRING) AS cgpa_text FROM student;
CAST to INT	Convert CGPA (decimal) into integer (drops decimals).	SELECT sname, CAST(cgpa AS INT) AS cgpa_int FROM student;
CAST to DECIMAL	Convert regno (INT) into DECIMAL(10,2).	SELECT regno, CAST(regno AS DECIMAL(10,2)) AS reg_decimal FROM student;
CAST to DATE	Convert a string into DATE explicitly.	SELECT CAST('2005-04-12' AS DATE) AS birth_date;
CAST to TIMESTAMP	Turn a date-time string into TIMESTAMP.	SELECT CAST('2025-09-22 14:30:00' AS TIMESTAMP);
CAST JSON → STRING	Convert JSON to string for searching.	SELECT sname, CAST(address AS STRING) AS addr_text FROM student;
CAST STRING → JSON	Convert text literal into JSON type.	SELECT CAST('{"city":"Chennai"}' AS JSON) AS addr;
CAST to DOUBLE	Convert integer marks element into DOUBLE precision.	SELECT sid, CAST(marks[1] AS DOUBLE) AS first_mark FROM student;

Keyword	Description	Example
CAST to BOOLEAN	Convert integer expressions into boolean.	SELECT regno, CAST((cgpa > 8) AS BOOLEAN) AS high_performer FROM student;

Mathematics Functions

Keyword / Function	Description	Example
+ - * / %	Arithmetic operators; / on integers truncates; % is modulo (remainder).	SELECT regno, regno/2 AS half, regno%2 AS rem FROM student;
& ^ ~ << >>	Bitwise AND/OR/XOR/NOT and bit shifts (integers only).	SELECT regno, regno & 15 AS and15, regno ^ 3 AS xor3, regno << 1 AS shl FROM student;
abs(x)	Absolute value.	SELECT sname, abs(cgpa-9.0) AS diff FROM student;
sign(x)	Returns -1, 0, or 1 based on sign.	SELECT sname, sign(cgpa-8.0) AS above8 FROM student;
ceil(x) / ceiling(x)	Rounds up to nearest integer.	SELECT sname, ceil(cgpa) AS up FROM student;
floor(x)	Rounds down to nearest integer.	SELECT sname, floor(cgpa) AS down FROM student;
round(x,d)	Round to d decimals.	SELECT sname, round(cgpa,1) AS cg FROM student;
power(x,y)	x raised to power y.	SELECT regno, power(2,2) AS four FROM student LIMIT 1;
sqrt(x)	Square root.	SELECT sqrt(2.0) AS root2;
cbrt(x)	Cube root.	SELECT cbrt(27.0) AS cube_root;
exp(x)	Exponential e^x.	SELECT exp(1) AS e;
ln(x) / log(x)	Natural logarithm (same function).	SELECT ln(2.0) AS ln2, log(2.0) AS log2n;
log10(x) / log2(x)	Base-10 and base-2 logs.	SELECT log10(100.0), log2(64.0);
log(b,x)	Logarithm base b. (Arg order is b,x in modern releases.)	SELECT log(2,64.0) AS log2_64;
mod(x,y)	Modulo function (works with decimals too).	SELECT mod(5.0,2.1) AS m;

Keyword / Function	Description	Example
greatest(x,y) / least(x,y)	Return higher/lower of two values (same as sql_max/sql_min).	SELECT greatest(cgpa,8.0), least(cgpa,8.0) FROM student;
rand() / rand(seed)	Random 32-bit integer; optional seed.	SELECT sname, rand() AS r FROM student;
nextafter(x,y)	Next representable float from x toward y.	SELECT nextafter(5.999999999999999,12.1);
bit_and/or/xor/not	Bitwise functions (function form).	SELECT bit_and(91,15), bit_or(32,3), bit_xor(17,5), bit_not(1);
left_shift/right_shift	Bit shifts (function form).	SELECT left_shift(1,4), right_shift(16,2);
degrees(rad) / radians(deg)	Convert rad↔deg; pi() constant.	SELECT degrees(pi()/2), radians(180), pi();
sys.ms_round(x,prc,trunc)	Round to prc decimals; truncate control.	SELECT sys.ms_round(1.2359,2,0);
sys.ms_trunc(x,prc)	Truncate to prc decimals.	SELECT sys.ms_trunc(1.2359,2);

String Functions

Keyword	Description	Example
LENGTH(str)	Returns number of characters in a string.	SELECT sname, LENGTH(sname) AS name_len FROM student;
UPPER(str)	Converts all characters to uppercase.	SELECT UPPER(sname) AS caps_name FROM student;
LOWER(str)	Converts all characters to lowercase.	SELECT LOWER(sname) AS small_name FROM student;
INITCAP(str)	Capitalizes first letter of each word.	SELECT INITCAP(LOWER(sname)) AS proper FROM student;
CONCAT(str1,str2)	Concatenates two strings.	SELECT CONCAT(sname, branch) AS details FROM student;
SUBSTRING(str FROM x FOR y)	Extracts substring starting at position x with length y.	SELECT SUBSTRING(sname FROM 1 FOR 5) AS short_name FROM student;
POSITION(substr IN str)	Finds index of substring in string.	SELECT POSITION('a' IN sname) AS pos FROM student;

Keyword	Description	Example
TRIM([chars FROM] str)	Removes spaces (or specified chars) from ends.	SELECT TRIM(BOTH ' ' FROM ' Delhi ');
REPLACE(str, old, new)	Replaces substring occurrences.	SELECT REPLACE(sname,'a','@') AS funny FROM student;
REPEAT(str,n)	Repeats string n times.	SELECT REPEAT('CS',3) AS code FROM student;

Generator Functions

Keyword	Description	Example
generate_series(start, stop)	Generates a series of integers from start to stop inclusive, step = 1.	SELECT * FROM generate_series(1,5);
generate_series(start, stop, step)	Generates integers with a custom step.	SELECT * FROM generate_series(2,10,2);
generate_series + aggregates	Combine with aggregates for numeric tests.	SELECT AVG(g) FROM generate_series(1,100) AS g;

View in MonetDB

Feature	Purpose / Description	Example
CREATE VIEW	Defines a virtual table based on a SELECT query. The view does not store data physically.	CREATE VIEW Maxmarks AS SELECT regno, name, cgpa FROM student WHERE cgpa > 9;
SHOW VIEW	View display	SELECT * FROM Maxmarks;
DROP VIEW	Removes an existing view from the database.	DROP VIEW Maxmarks;
CREATE INDEX	Improves query performance by creating an index on one or more columns.	CREATE INDEX idx_stu_cgpa ON student(cgpa);
DROP INDEX	Deletes an existing index.	DROP INDEX idx_stu_cgpa;

Indexes in MonetDB

Keyword	Description	Example
CREATE INDEX	Create a simple index on a column.	CREATE INDEX idx_student_name ON student(sname);
CREATE UNIQUE INDEX	Ensure values are unique (like alternate key).	CREATE UNIQUE INDEX idx_regno_unique ON student(regno);
CREATE INDEX on multiple cols	Index on combination of columns.	CREATE INDEX idx_branch_cgpa ON student(branch, cgpa);
CREATE INDEX with schema name	Create index with fully qualified name.	CREATE INDEX university.idx_city ON student((CAST(address AS STRING)));
DROP INDEX	Remove an index by name.	DROP INDEX idx_student_name;
DROP INDEX IF EXISTS	Prevent error if index missing.	DROP INDEX IF EXISTS idx_branch_cgpa;
List indexes	Query system catalog for existing indexes.	SELECT name, table_id FROM sys.idxs;
Index + query	Index helps speed up filtering queries.	SELECT sname FROM student WHERE sname LIKE 'A%';
Index on expression	Create index on computed expression.	CREATE INDEX idx_year ON student(EXTRACT(YEAR FROM dob));

Nested Queries (subqueries) in MonetDB

Keyword	Description	Example
Subquery in WHERE (scalar)	Compare with a single value from a subquery.	SELECT sname, cgpa FROM student WHERE cgpa > (SELECT AVG(cgpa) FROM student);
Subquery in SELECT	Project derived value alongside each row.	SELECT sname, (SELECT MAX(cgpa) FROM student) AS max_cgpa FROM student;
Subquery in FROM (inline view)	Use a subquery as a virtual table.	SELECT branch, AVG(cgpa) FROM (SELECT * FROM student WHERE cgpa>=8) AS top GROUP BY branch;
IN with subquery	Match values against set from subquery.	SELECT sname FROM student WHERE branch IN (SELECT DISTINCT branch FROM student WHERE cgpa>8.5);
EXISTS subquery	Check if related rows exist.	SELECT sname FROM student s WHERE EXISTS (SELECT 1 FROM student t WHERE t.branch=s.branch AND t.cgpa>9);
NOT EXISTS	Opposite of EXISTS; returns when no match found.	SELECT sname FROM student s WHERE NOT EXISTS (SELECT 1 FROM student t WHERE t.branch=s.branch AND t.cgpa>9);
ANY / SOME	Compare with any value from subquery.	SELECT sname, cgpa FROM student WHERE cgpa > ANY (SELECT cgpa FROM student WHERE branch='ECE');
ALL	Compare with all values from subquery.	SELECT sname, cgpa FROM student WHERE cgpa > ALL (SELECT cgpa FROM student WHERE branch='MECH');
Nested update	Update rows using subquery result.	UPDATE student SET branch='TOPPER' WHERE cgpa=(SELECT MAX(cgpa) FROM student);
Correlated subquery	Subquery depends on outer row.	SELECT s1.sname, s1.cgpa FROM student s1 WHERE s1.cgpa >= (SELECT AVG(s2.cgpa) FROM student s2 WHERE s2.branch=s1.branch);

Joins in MonetDB:

course_id	cname	branch	regno
1	Database Systems	CSE	25001
2	Digital Electronics	ECE	25002
3	Electrical Machines	EEE	25003
4	Operating Systems	IT	25004
5	Thermodynamics	MECH	25005

Operation	Description	Example Query
Create course table with foreign key	Creates a course table linked to the student table via regno	CREATE TABLE course (course_id SERIAL, cname VARCHAR(100) NOT NULL, branch VARCHAR(50), regno INT, FOREIGN KEY (regno) REFERENCES student(regno));
Insert CSE course	Inserts a record for Database Systems offered to regno 101	INSERT INTO course (cname, branch, regno) SELECT 'Database Systems', 'CSE', regno FROM student WHERE sname = 'Rohan Mehta' UNION ALL SELECT 'Digital Electronics', 'ECE', regno FROM student WHERE sname = 'Ananya Gupta' UNION ALL SELECT 'Electrical Machines', 'EEE', regno FROM student WHERE sname = 'Vikram Reddy' UNION ALL SELECT 'Operating Systems', 'IT', regno FROM student WHERE sname = 'Priya Nair' UNION ALL SELECT 'Thermodynamics', 'MECH', regno FROM student WHERE sname = 'Karthik Sharma';
Show courses	Show all courses in ascending order	SELECT * FROM course ORDER BY course_id;

Foreign key violation

INSERT INTO course (cname, branch, regno) VALUES ('Artificial Intelligence', 'AIML', 26001), ('Cloud Computing', 'CLOUD', 26002);

INSERT INTO course (cname, branch, regno) VALUES ('Artificial Intelligence', 'AIML', 25001), ('Cloud Computing', 'CLOUD', 25002);

Insert two students:

INSERT INTO student (sname, dob, cgpa, address, branch) VALUES ('Aditi Rao', DATE '2006-09-15', 8.60, CAST({'street':"Anna Nagar","city":"Chennai","state":"TN"} AS JSON), 'AI'), ('Harish Patel', DATE '2005-05-27', 7.95, CAST({'street':"Law Garden","city":"Ahmedabad","state":"GJ"} AS JSON), 'CIVIL');

Insert one row in course:

INSERT INTO course (cname, branch, regno) VALUES ('Quantum Computing', 'PHY', NULL);

Join Queries:

Keyword	Description	Example
INNER JOIN	Returns rows where values match in both tables.	SELECT s.sname, c.cname FROM student s INNER JOIN course c ON s.regno=c.regno;
LEFT JOIN	Returns all students, with courses if available; otherwise NULL.	SELECT s.sname, c.cname FROM student s LEFT JOIN course c ON s.regno=c.regno;
RIGHT JOIN	Returns all courses, with matching students if available.	SELECT s.sname, c.cname FROM student s RIGHT JOIN course c ON s.regno=c.regno;
FULL OUTER JOIN	Returns all students and all courses, matches where possible.	SELECT s.sname, c.cname FROM student s FULL OUTER JOIN course c ON s.regno=c.regno;
CROSS JOIN	Returns Cartesian product of students $\times$ courses.	SELECT s.sname, c.cname FROM student s CROSS JOIN course c;
NATURAL JOIN	Joins automatically on columns with same name (sid).	SELECT sname, cname FROM student NATURAL JOIN course;
SELF JOIN	Student table joined with itself (e.g., pair students in same branch).	INSERT INTO student (sname, dob, cgpa, address, branch) VALUES ('Divya Menon', DATE '2006-03-20', 8.25, CAST({'street':"MG Road","city":"Bengaluru","state":"KA"} AS JSON), 'CSE'), ('Rahul Iyer', DATE '2006-06-10', 7.90, CAST({'street':"Guindy","city":"Chennai","state":"TN"} AS JSON), 'ECE'); SELECT a.sname AS st1, b.sname AS st2, a.branch FROM student a JOIN student b ON a.branch=b.branch AND a.regno<b.regno;

Keyword	Description	Example
JOIN with inequality	Join with non-equality condition.	SELECT s.sname, c.cname FROM student s JOIN course c ON s.regno <> c.regno;
JOIN with USING	Shortcut when both tables have same column name.	SELECT sname, cname FROM student JOIN course USING(regno);
Multi-table JOIN	Join more than two tables (extend if more exist).	CREATE TABLE department (dept_id SERIAL, dept_name VARCHAR(100) NOT NULL, branch VARCHAR(50) UNIQUE); INSERT INTO department (dept_name, branch) VALUES ('Computer Science and Engineering', 'CSE'), ('Electronics and Communication Engineering', 'ECE'), ('Electrical and Electronics Engineering', 'EEE'), ('Information Technology', 'IT'), ('Mechanical Engineering', 'MECH'), ('Artificial Intelligence and Machine Learning', 'AIML'), ('Cloud Technology', 'CLOUD'), ('Civil Engineering', 'CIVIL'); SELECT s.sname, c.cname, d.dept_name FROM student s JOIN course c ON s.regno=c.regno JOIN department d ON s.branch=d.branch;

Admin Commands in MonetDB

Keyword	Description	Example	Output
CREATE USER	Create a new database user with password.	CREATE USER alice WITH PASSWORD 'alice123' NAME 'Alice Kumar' SCHEMA university;	operation successful
CREATE ROLE	Create a new role (group of privileges).	CREATE ROLE faculty;	operation successful
GRANT (to user)	Give user access to a schema or object.	GRANT SELECT, INSERT ON student TO alice;	operation successful
GRANT (role to user)	Assign a role to a user.	GRANT faculty TO alice;	operation successful
REVOKE (privilege)	Remove specific privileges from a user.	REVOKE INSERT ON student FROM alice;	operation successful
REVOKE (role)	Remove a role from a user.	REVOKE faculty FROM alice;	operation successful

Keyword	Description	Example	Output
ALTER USER (password)	Change a user's password.	ALTER USER alice WITH PASSWORD 'newpass123';	operation successful
ALTER USER (schema)	Change default schema of a user.	ALTER USER alice SET SCHEMA sys;	operation successful
DROP USER	Remove a user account completely.	DROP USER IF EXISTS alice;	operation successful
DROP ROLE	Delete a role from the system.	DROP ROLE IF EXISTS faculty;	operation successful